Министерство науки и высшего образования РФ
ФГАОУ ВПО
Национальный исследовательский технологический университет
«МИСИС»

Институт информационных технологий и компьютерных наук (ИТКН)

Кафедра инфокоммуникационных технологий (ИКТ)

Отчет по лабораторной работе № 5
по дисциплине «Объектно-ориентированное программирование»

на тему «Механизмы наследования и полиморфизма в Си#»

Выполнил:
студент группы БИВТ-25-1
Кирюшин А. А.
Проверил:
доцент каф. ИКТ
Стучилин В.В.

Москва, 2026

1. ЦЕЛЬ РАБОТЫ

Приобрести навыки работы с механизмом наследования и полиморфизма в языке программирования Си#.

2. ЗАДАНИЕ НА ЛАБОРАТОРНУЮ РАБОТУ

В рамках лабораторной работы были решены следующие задачи:

- Реализовать абстрактный базовый класс `GeometricShape`
- Реализовать класс `Square`
- Реализовать класс `Circle`
- Реализовать класс `Triangle`
- Разработать графический интерфейс

5. ЛИСТИНГ ПРОГРАММЫ

Абстрактный базовый класс:

```
using System;

namespace GeometryGUI
{
    public abstract class GeometricShape
    {
        protected double centerX;
        protected double centerY;
        protected string color;
        protected double size;
        protected string shapeName;

        public double CenterX
        {
            get { return centerX; }
            set { centerX = value; }
        }

        public double CenterY
        {
            get { return centerY; }
            set { centerY = value; }
        }

        public string Color
        {
            get { return color; }
            set { color = value; }
        }

        public double Size
        {
            get { return size; }
            set { size = value; }
        }
```

```
        public string ShapeName
        {
            get { return shapeName; }
        }

        public abstract double CalculateArea();
        public abstract void Draw();
        public abstract bool ContainsPoint(double x,
double y);

        public virtual void SelectShape()
        {
            Console.WriteLine($"Фигура: {shapeName}");
            Console.WriteLine($"Площадь:
{CalculateArea():F2}");
            Console.WriteLine($"Центр: ({centerX:F1},
{centerY:F1})");
            Console.WriteLine($"Размер: {size:F1}");
            Console.WriteLine($"Цвет: {color}");
        }
    }
}
```

Класс квадрата:

```
using System;

namespace GeometryGUI
{
    public class Square : GeometricShape
    {
        public double Side => size;

        public Square(double centerX, double centerY,
double side, string color)
        {
            this.centerX = centerX;
            this.centerY = centerY;
            this.size = side;
            this.color = color;
            this.shapeName = "Квадрат";
        }

        public override double CalculateArea()
        {
            return size * size;
        }

        public override void Draw()
        {
            Console.WriteLine($"□ {shapeName} - центр:
({centerX:F1}, {centerY:F1}), сторона: {size:F1}, цвет:
{color}");
        }

        public override bool ContainsPoint(double x,
double y)
        {
            double halfSide = size / 2;
            return Math.Abs(x - centerX) <= halfSide &&
Math.Abs(y - centerY) <= halfSide;
        }
    }
}
```

Класс круга:

```
using System;

namespace GeometryGUI
{
    public class Circle : GeometricShape
    {
        public double Diameter => size;

        public Circle(double centerX, double centerY,
double diameter, string color)
        {
            this.centerX = centerX;
            this.centerY = centerY;
            this.size = diameter;
            this.color = color;
            this.shapeName = "Круг";
        }

        public override double CalculateArea()
        {
            double radius = size / 2;
            return Math.PI * radius * radius;
        }

        public override void Draw()
        {
            Console.WriteLine($"○ {shapeName} - центр:
({centerX:F1}, {centerY:F1}), диаметр: {size:F1}, цвет:
{color}");
        }

        public override bool ContainsPoint(double x,
double y)
        {
            double radius = size / 2;
            double distance = Math.Sqrt(Math.Pow(x -
centerX, 2) + Math.Pow(y - centerY, 2));
            return distance <= radius;
        }
    }
```

Класс треугольника

```
using System;

namespace GeometryGUI
{
    public class Triangle : GeometricShape
    {
        public double Height => size;

        public Triangle(double centerX, double centerY,
double height, string color)
        {
            this.centerX = centerX;
            this.centerY = centerY;
            this.size = height;
            this.color = color;
            this.shapeName = "Треугольник";
        }

        public override double CalculateArea()
        {
            return (size * size * Math.Sqrt(3)) / 4;
        }

        public override void Draw()
        {
            Console.WriteLine($"Δ {shapeName} - центр:
({centerX:F1}, {centerY:F1}), высота: {size:F1}, цвет:
{color}");
        }

        public override bool ContainsPoint(double x,
double y)
        {
            double radius = size / Math.Sqrt(3);
            double distance = Math.Sqrt(Math.Pow(x -
centerX, 2) + Math.Pow(y - centerY, 2));
            return distance <= radius;
        }
```

Графический интерфейс:

```
using Avalonia.Controls;
using Avalonia.Controls.Shapes;
using Avalonia.Input;
using Avalonia.Interactivity;
using Avalonia.Media;
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;

namespace GeometryGUI.Views
{
    public partial class MainWindow : Window
    {
        private List<GeometricShape> shapes = new
List<GeometricShape>();
        private GeometricShape? selectedShape = null;
        private Random random = new Random();
        private string[] colors = { "Red", "Blue",
"Green", "Orange", "Purple", "Brown", "Pink", "Cyan" };

        public MainWindow()
        {
            InitializeComponent();
        }

        private void OnCreateRandomClick(object?
sender, RoutedEventArgs e)
        {
            double canvasWidth =
DrawingCanvas.Bounds.Width;
            double canvasHeight =
DrawingCanvas.Bounds.Height;

            if (canvasWidth <= 0 || canvasHeight <= 0)
            {
                canvasWidth = 600;
                canvasHeight = 600;
            }
```

```
            int shapeType = random.Next(0, 3);
            double centerX = random.Next(100,
(int)canvasWidth - 100);
            double centerY = random.Next(100,
(int)canvasHeight - 100);
            string color =
colors[random.Next(colors.Length)];

            GeometricShape shape;
            switch (shapeType)
            {
                case 0:
                    double side = random.Next(30, 100);
                    shape = new Square(centerX,
centerY, side, color);
                    break;
                case 1:
                    double diameter = random.Next(40,
120);
                    shape = new Circle(centerX,
centerY, diameter, color);
                    break;
                default:
                    double height = random.Next(40,
100);
                    shape = new Triangle(centerX,
centerY, height, color);
                    break;
            }

            shapes.Add(shape);
            DrawShape(shape);
            StatusLabel.Text = $"Создана фигура:
{shape.GetType().Name}";
        }

        private void DrawShape(GeometricShape shape)
        {
            Shape visual;
            IBrush brush = new
SolidColorBrush(ParseColor(shape.Color));
            IBrush stroke = Brushes.Black;
```

```
            if (shape is Square square)
            {
                visual = new Rectangle
                {
                    Width = square.Side,
                    Height = square.Side,
                    Fill = brush,
                    Stroke = stroke,
                    StrokeThickness = 2
                };
                Canvas.SetLeft(visual, shape.CenterX -
square.Side / 2);
                Canvas.SetTop(visual, shape.CenterY -
square.Side / 2);
            }
            else if (shape is Circle circle)
            {
                visual = new Ellipse
                {
                    Width = circle.Diameter,
                    Height = circle.Diameter,
                    Fill = brush,
                    Stroke = stroke,
                    StrokeThickness = 2
                };
                Canvas.SetLeft(visual, shape.CenterX -
circle.Diameter / 2);
                Canvas.SetTop(visual, shape.CenterY -
circle.Diameter / 2);
            }
            else if (shape is Triangle triangle)
            {
                double halfBase = triangle.Height / 2;
                Polygon polygon = new Polygon
                {
                    Fill = brush,
                    Stroke = stroke,
                    StrokeThickness = 2,
                    Points = new
Avalonia.Collections.AvaloniaList<Avalonia.Point>
                    {
                        new Avalonia.Point(0,
-triangle.Height * 2.0 / 3.0),
```

```
                        new Avalonia.Point(-halfBase,
triangle.Height / 3.0),
                        new Avalonia.Point(halfBase,
triangle.Height / 3.0)
                    }
                };
                visual = polygon;
                Canvas.SetLeft(visual, shape.CenterX);
                Canvas.SetTop(visual, shape.CenterY);
            }
            else
            {
                return;
            }

            visual.Tag = shape;
            DrawingCanvas.Children.Add(visual);
        }

        private void OnCanvasPointerPressed(object?
sender, PointerPressedEventArgs e)
        {
            var point = e.GetPosition(DrawingCanvas);
            double x = point.X;
            double y = point.Y;

            selectedShape = null;
            foreach (var shape in shapes)
            {
                if (shape.ContainsPoint(x, y))
                {
                    selectedShape = shape;
                    break;
                }
            }

            if (selectedShape != null)
            {
                ShapeInfoLabel.Text = $"Тип:
{selectedShape.GetType().Name}\n" +
                                        $"Центр:
({selectedShape.CenterX:F1},
{selectedShape.CenterY:F1})\n" +
```

```
                                        $"Цвет:
{selectedShape.Color}\n" +
                                        $"Площадь:
{selectedShape.CalculateArea():F2}";
                selectedShape.SelectShape();
            }
            else
            {
                ShapeInfoLabel.Text = "Выберите фигуру
на холсте";
            }
        }

        private void OnCheckPointClick(object? sender,
RoutedEventArgs e)
        {
            if (!double.TryParse(PointXTextBox.Text,
out double x) ||
                !double.TryParse(PointYTextBox.Text,
out double y))
            {
                PointResultLabel.Text = "Ошибка:
введите корректные координаты";
                PointResultLabel.Foreground =
Brushes.Red;
                return;
            }

            if (selectedShape == null)
            {
                PointResultLabel.Text = "Выберите
фигуру на холсте";
                PointResultLabel.Foreground =
Brushes.Orange;
                return;
            }

            bool inside =
selectedShape.ContainsPoint(x, y);
            PointResultLabel.Text = inside
                ? $"Точка ({x:F1}, {y:F1}) ВНУТРИ
выбранной фигуры"
```

```
                : $"Точка ({x:F1}, {y:F1}) ВНЕ
выбранной фигуры";
            PointResultLabel.Foreground = inside ?
Brushes.Green : Brushes.Red;
        }

        private void OnClearAllClick(object? sender,
RoutedEventArgs e)
        {
            shapes.Clear();
            DrawingCanvas.Children.Clear();
            selectedShape = null;
            ShapeInfoLabel.Text = "Выберите фигуру на
холсте";
            StatusLabel.Text = "Холст очищен";
        }
private void OnSaveClick(object? sender,
RoutedEventArgs e)
        {
            try
            {
                string filePath = "shapes.txt";
                using (StreamWriter writer = new
StreamWriter(filePath))
                {
                    foreach (var shape in shapes)
                    {
                        if (shape is Square square)
                        {
                            writer.WriteLine($"Square|
{square.CenterX}|{square.CenterY}|{square.Side}|
{square.Color}");
                        }
                        else if (shape is Circle
circle)
                        {
                            writer.WriteLine($"Circle|
{circle.CenterX}|{circle.CenterY}|{circle.Diameter}|
{circle.Color}");
                        }
                        else if (shape is Triangle
triangle)
                        {
```

```
writer.WriteLine($"Triangle|{triangle.CenterX}|
{triangle.CenterY}|{triangle.Height}|
{triangle.Color}");
                        }
                    }
                }
                StatusLabel.Text = $"Сохранено
{shapes.Count} фигур в файл {filePath}";
                StatusLabel.Foreground = Brushes.Green;
            }
            catch (Exception ex)
            {
                StatusLabel.Text = $"Ошибка сохранения:
{ex.Message}";
                StatusLabel.Foreground = Brushes.Red;
            }
        }

        private void OnLoadClick(object? sender,
RoutedEventArgs e)
        {
            try
            {
                string filePath = "shapes.txt";
                if (!File.Exists(filePath))
                {
                    StatusLabel.Text = "Файл shapes.txt
не найден";
                    StatusLabel.Foreground =
Brushes.Orange;
                    return;
                }

                shapes.Clear();
                DrawingCanvas.Children.Clear();

                string[] lines =
File.ReadAllLines(filePath);
                foreach (string line in lines)
                {
                    string[] parts = line.Split('|');
                    if (parts.Length < 5) continue;
```

```
                    string type = parts[0];
                    double centerX =
double.Parse(parts[1]);
                    double centerY =
double.Parse(parts[2]);
                    double size =
double.Parse(parts[3]);
                    string color = parts[4];

                    GeometricShape shape;
                    switch (type)
                    {
                        case "Square":
                            shape = new Square(centerX,
centerY, size, color);
                            break;
                        case "Circle":
                            shape = new Circle(centerX,
centerY, size, color);
                            break;
                        case "Triangle":
                            shape = new
Triangle(centerX, centerY, size, color);
                            break;
                        default:
                            continue;
                    }

                    shapes.Add(shape);
                    DrawShape(shape);
                }

                StatusLabel.Text = $"Загружено
{shapes.Count} фигур из файла {filePath}";
                StatusLabel.Foreground = Brushes.Green;
            }
            catch (Exception ex)
            {
                StatusLabel.Text = $"Ошибка загрузки:
{ex.Message}";
                StatusLabel.Foreground = Brushes.Red;
            }
```

```
        }

        private Color ParseColor(string colorName)
        {
            return colorName.ToLower() switch
            {
                "red" => Colors.Red,
                "blue" => Colors.Blue,
                "green" => Colors.Green,
                "orange" => Colors.Orange,
                "purple" => Colors.Purple,
                "brown" => Colors.Brown,
                "pink" => Colors.Pink,
                "cyan" => Colors.Cyan,
                _ => Colors.Gray
            };
        }
    }
}
```

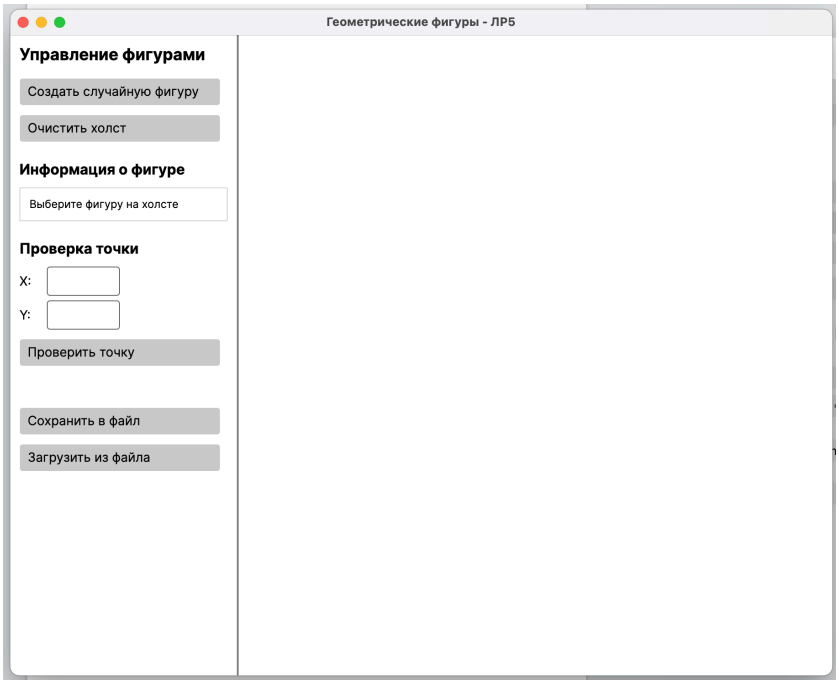
Шаблон:

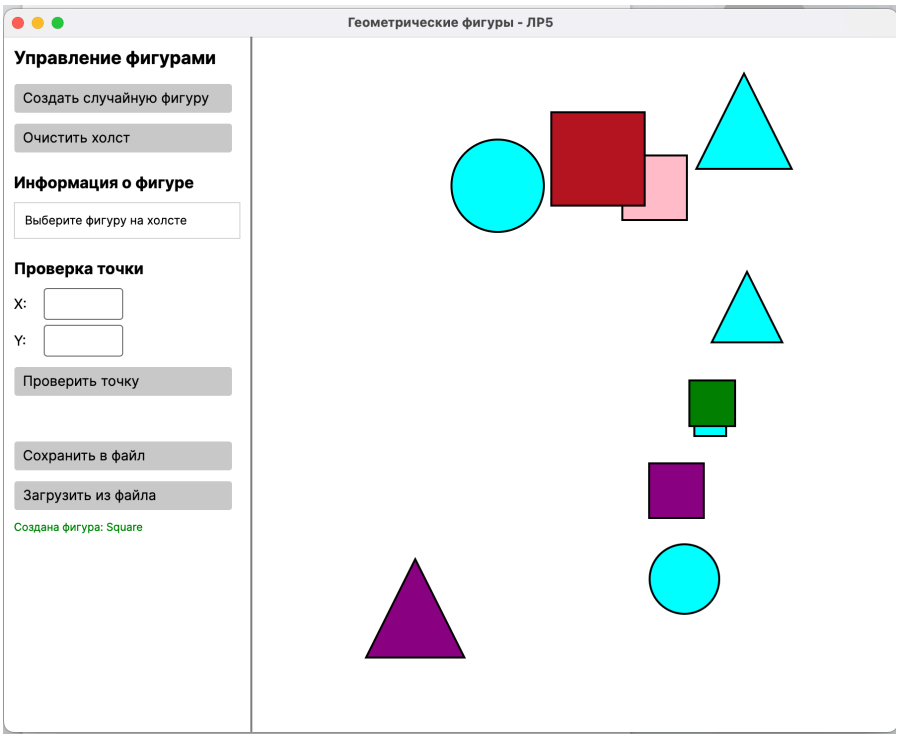
```
chemas.microsoft.com/winfx/2006/xaml"
        x:Class="GeometryGUI.Views.MainWindow"
        Title="Геометрические фигуры - ЛР5"
        Width="900" Height="700">

    <Grid>
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="250"/>
            <ColumnDefinition Width="*"/>
        </Grid.ColumnDefinitions>

        <!-- Левая панель управления -->
        <Border Grid.Column="0" BorderBrush="Gray"
BorderThickness="0,0,2,0" Padding="10">
            <StackPanel>
                <TextBlock Text="Управление фигурами"
FontSize="18" FontWeight="Bold" Margin="0,0,0,15"/>

                <Button Name="CreateRandomButton"
Content="Создать случайную фигуру"
                        Width="220" Margin="0,0,0,10"
Click="OnCreateRandomClick"/>

                <Button Name="ClearAllButton"
Content="Очистить холст"
                        Width="220" Margin="0,0,0,20"
Click="OnClearAllClick"/>

                <TextBlock Text="Информация о фигуре"
FontSize="16" FontWeight="Bold" Margin="0,0,0,10"/>

                <Border BorderBrush="LightGray"
BorderThickness="1" Padding="10" Margin="0,0,0,10">
                    <StackPanel>
                        <TextBlock
Name="ShapeInfoLabel" Text="Выберите фигуру на холсте"
                                   TextWrapping="Wrap"
FontSize="12"/>
                    </StackPanel>
```

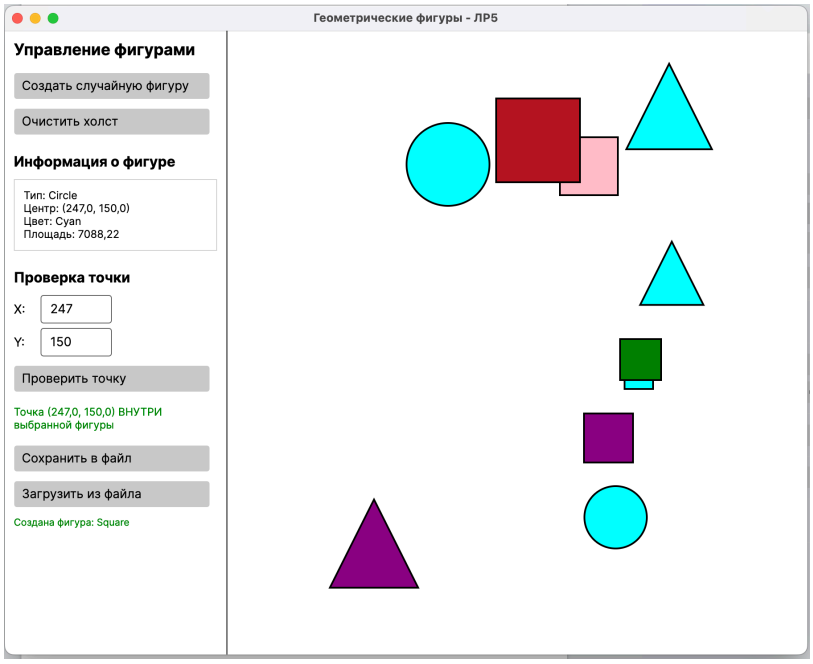
```
                </Border>

                <TextBlock Text="Проверка точки"
FontSize="16" FontWeight="Bold" Margin="0,10,0,10"/>

                <StackPanel Orientation="Horizontal"
Margin="0,0,0,5">
                    <TextBlock Text="X:" Width="30"
VerticalAlignment="Center"/>
                    <TextBox Name="PointXTextBox"
Width="80"/>
                </StackPanel>

                <StackPanel Orientation="Horizontal"
Margin="0,0,0,10">
                    <TextBlock Text="Y:" Width="30"
VerticalAlignment="Center"/>
                    <TextBox Name="PointYTextBox"
Width="80"/>
                </StackPanel>

                <Button Name="CheckPointButton"
Content="Проверить точку"
                        Width="220" Margin="0,0,0,10"
Click="OnCheckPointClick"/>

                <TextBlock Name="PointResultLabel"
Text="" TextWrapping="Wrap"
                           FontSize="12"
Foreground="DarkBlue" Margin="0,5,0,15"/>

                <Button Name="SaveButton"
Content="Сохранить в файл"
                        Width="220" Margin="0,0,0,10"
Click="OnSaveClick"/>

                <Button Name="LoadButton"
Content="Загрузить из файла"
                        Width="220" Click="OnLoadClick"/
>

                <TextBlock Name="StatusLabel" Text=""
TextWrapping="Wrap"
```

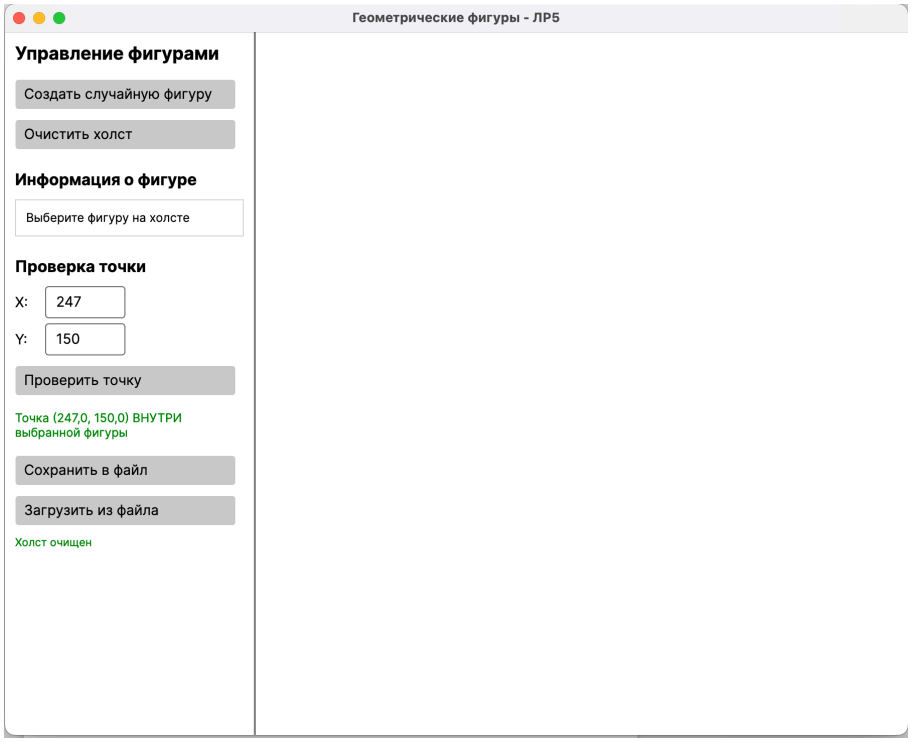
```
                            FontSize="11"
Foreground="Green" Margin="0,10,0,0"/>
            </StackPanel>
        </Border>

        <!-- Правая панель с Canvas -->
        <Border Grid.Column="1" Background="White">
            <Canvas Name="DrawingCanvas"
Background="White"

PointerPressed="OnCanvasPointerPressed"/>
        </Border>
    </Grid>

</Window>
```

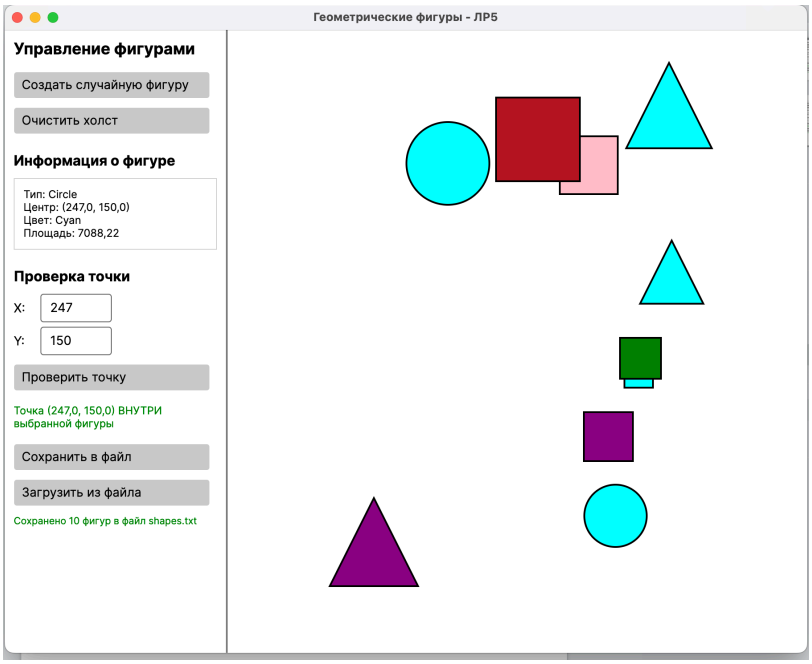
Выполнение:

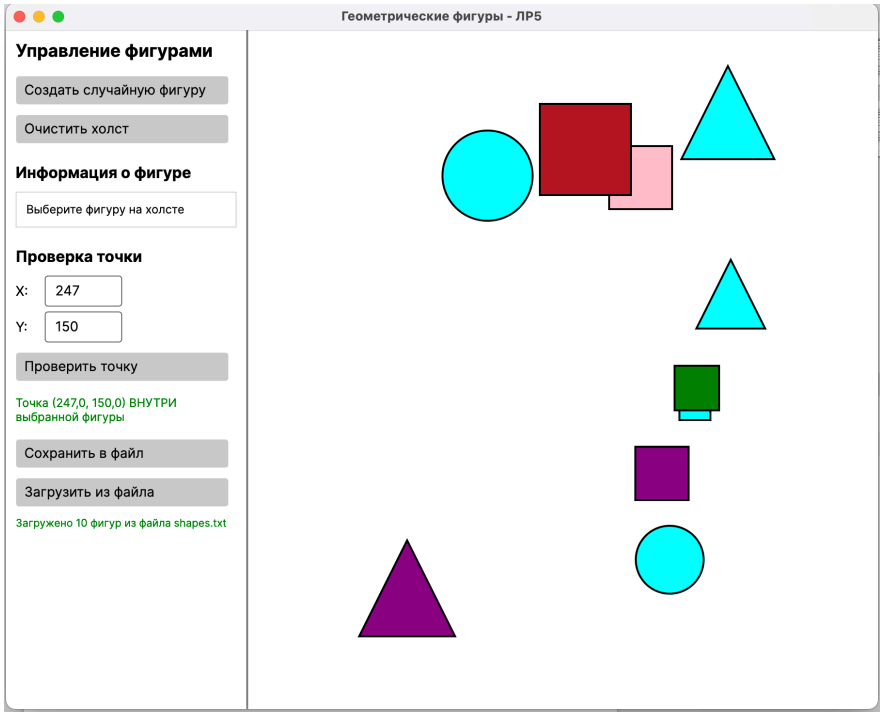

Геометрические фигуры - ЛР5
Управление фигурами
Создать случайную фигуру
Очистить холст
Информация о фигуре
Выберите фигуру на холсте
Проверка точки
X:
Y:
Проверить точку
Сохранить в файл
Загрузить из файла


Геометрические фигуры - ЛР5
Управление фигурами
Создать случайную фигуру
Очистить холст
Информация о фигуре
Выберите фигуру на холсте
Проверка точки
X:
Y:
Проверить точку
Сохранить в файл
Загрузить из файла
Создана фигура: Square


Геометрические фигуры - ЛР5
Управление фигурами
Создать случайную фигуру
Очистить холст
Информация о фигуре
Тип: Circle
Центр: (247,0, 150,0)
Цвет: Cyan
Площадь: 7088,22
Проверка точки
X:
247
Y:
150
Проверить точку
Точка (247,0, 150,0) ВНУТРИ выбранной фигуры
Сохранить в файл
Загрузить из файла
Создана фигура: Square


Геометрические фигуры - ЛР5
Управление фигурами
Создать случайную фигуру
Очистить холст
Информация о фигуре
Выберите фигуру на холсте
Проверка точки
X:
247
Y:
150
Проверить точку
Точка (247,0, 150,0) ВНУТРИ выбранной фигуры
Сохранить в файл
Загрузить из файла
Холст очищен


Геометрические фигуры - ЛР5
Управление фигурами
Создать случайную фигуру
Очистить холст
Информация о фигуре
Тип: Circle
Центр: (247,0, 150,0)
Цвет: Cyan
Площадь: 7088,22
Проверка точки
X:
247
Y:
150
Проверить точку
Точка (247,0, 150,0) ВНУТРИ выбранной фигуры
Сохранить в файл
Загрузить из файла
Сохранено 10 фигур в файл shapes.txt


Геометрические фигуры - ЛР5
Управление фигурами
Создать случайную фигуру
Очистить холст
Информация о фигуре
Выберите фигуру на холсте
Проверка точки
X:
247
Y:
150
Проверить точку
Точка (247,0, 150,0) ВНУТРИ выбранной фигуры
Сохранить в файл
Загрузить из файла
Загружено 10 фигур из файла shapes.txt

7. ВЫВОД

В ходе выполнения лабораторной работы я успешно освоил ключевые механизмы объектно-ориентированного программирования на языке C#: наследование, полиморфизм, абстрактные классы и методы. Получил практический опыт создания иерархии классов с использованием абстрактного базового класса, реализации виртуальных и абстрактных методов, работы с коллекциями полиморфных объектов. Научился правильно применять модификаторы доступа для обеспечения инкапсуляции данных, использовать ключевое слово override для переопределения методов базового класса, работать с защищенными (protected) полями, которые доступны только наследникам.

Реализовал полную иерархию классов геометрических фигур с базовым абстрактным классом GeometricShape и тремя производными классами: Square (квадрат), Circle (круг) и Triangle (треугольник). Класс GeometricShape инкапсулирует общие для всех фигур свойства (координаты центра, цвет, размер, название) и объявляет абстрактные методы, которые должны быть реализованы в каждом производном классе: CalculateArea() для расчета площади, Draw() для отображения информации, ContainsPoint() для проверки принадлежности точки фигуре. Каждый производный класс реализует собственную специфичную логику: квадрат использует формулу $S = a^2$, круг вычисляет площадь по формуле $S = \pi \cdot r^2$, треугольник применяет формулу для равностороннего треугольника $S = (a^2 \cdot \sqrt{3})/4$. Освоил работу с математическими функциями Math.PI, Math.Sqrt, Math.Pow для выполнения геометрических вычислений.

Научился работать с полиморфными коллекциями типа List<GeometricShape>, где хранятся объекты разных производных классов через ссылку на базовый тип. Понял механизм позднего связывания (late binding), когда при вызове метода на объекте базового типа автоматически вызывается переопределенная версия этого метода из конкретного производного класса. Освоил использование оператора is для проверки типа объекта во время выполнения программы, что позволяет безопасно выполнять операции, специфичные для конкретного типа фигуры. Применил паттерн проектирования "Шаблонный метод" через определение абстрактных методов в базовом классе, которые реализуются в производных классах по-разному, обеспечивая гибкость и расширяемость системы.

Создал полнофункциональное графическое приложение на базе кроссплатформенного UI-фреймворка Avalonia, которое демонстрирует визуализацию геометрических фигур на холсте Canvas. Разработал XAML-разметку интерфейса с использованием Grid для разделения на две области: панель управления слева с кнопками, текстовыми полями и информационными метками, и Canvas справа для отрисовки фигур. Реализовал code-behind логику с обработчиками событий для создания случайных фигур, клика мыши по Canvas для выбора фигуры, проверки принадлежности точки с заданными координатами выбранной фигуре. Научился работать с визуальными элементами Avalonia: Rectangle для отрисовки квадратов, Ellipse для кругов, Polygon для треугольников,

применять к ним заливку (Fill) и обводку (Stroke) с использованием объектов IBrush и SolidColorBrush. Освоил методы позиционирования элементов на Canvas через Canvas.SetLeft() и Canvas.SetTop().

Реализовал функциональность сохранения и загрузки состояния приложения в текстовый файл с использованием классов StreamWriter и File для работы с файловой системой. Применил обработку исключений через конструкцию try-catch для корректной обработки ошибочных ситуаций: отсутствие файла, некорректный формат данных, проблемы с доступом к файлу. Научился правильно организовывать структуру данных для сериализации объектов в текстовый формат, используя символ разделителя для хранения свойств каждой фигуры в одной строке. Освоил десериализацию данных через разбор строк методом Split() и восстановление объектов нужного типа на основе сохраненной информации о типе фигуры.

Полученные навыки работы с наследованием, полиморфизмом и абстрактными классами являются фундаментальными для объектно-ориентированного программирования и создания масштабируемых приложений. Понял принципы проектирования классов с учетом общности и специфики объектов, научился выделять общий функционал в базовый класс и реализовывать специфичное поведение в производных классах. Освоил применение принципа открытости/закрытости (Open/Closed Principle): система открыта для расширения через добавление новых производных классов, но закрыта для модификации базового класса. Эти знания и практические навыки станут основой для разработки сложных объектно-ориентированных систем, применения паттернов проектирования, работы с интерфейсами и создания архитектурно правильных приложений на C#.